

# Getting Started Guide for Development

---

## Revised Page

| Version Number | Revision (Content) Description                                                           | Revision Time |
|----------------|------------------------------------------------------------------------------------------|---------------|
| V1.0.0         | C++ version launched, new document created.<br>(Corresponding SDK version number V3.0.0) | 2026/04/30    |

---

## 1. Compatibility and Requirements

Before starting to use the Dexterous Hand SDK, please confirm that the Development Environment meets the following requirements.

### 1.1 Hardware Requirements

| Item                    | Requirement                                             |
|-------------------------|---------------------------------------------------------|
| CPU Architecture        | x86 / x86_64                                            |
| Memory                  | 4 GB or above (recommended)                             |
| Communication Interface | USB, RS-485 (USB-to-485 converter), Ethernet (EtherCAT) |

### 1.2 Operating System Requirements

| Item             | Requirement                                 |
|------------------|---------------------------------------------|
| Operating System | Ubuntu 22.04 LTS                            |
| Kernel Version   | 5.15 or above (Ubuntu 22.04 default kernel) |

### 1.3 Communication Protocol Support

| Protocol   | Interface           | Typical Use                               |
|------------|---------------------|-------------------------------------------|
| Modbus-RTU | RS-485 / USB-to-485 | Academic research, small-scale deployment |
| USB        | USB-A / USB-C       | Direct-connect single-device debugging    |

---

## 2. Installation Guide

This section introduces the installation method of the GMH15 dexterous hand and solutions to common installation issues.

## 2.1 Operating Environment Requirements

| Item               | Requirement                                                        |
|--------------------|--------------------------------------------------------------------|
| Operating System   | Ubuntu 22.04 LTS                                                   |
| CPU                | 64-bit processor                                                   |
| Memory             | 4 GB RAM or above                                                  |
| Storage            | 20 GB available space or above                                     |
| C++                | C++17 standard                                                     |
| Other Dependencies | CMake >= 3.22, Boost 1.74.0, OpenCV 4.5.4, spdlog 1.9.2, fmt 8.1.1 |

## 2.2 Install DexHandSDK

- Install the DexHandSDK dependency library

```
# Dependencies (if they cannot all be installed at once, install them one by one)
sudo apt update && sudo apt install -y \
  build-essential \
  libboost-all-dev \
  libopencv-dev \
  libspdlog-dev \
  libfmt-dev \
  libgoogle-glog-dev \
  libopenblas-dev \
  libdlib-dev \
  libjsoncpp-dev \
  libfltk1.3-dev \
  libyaml-cpp-dev \
  python3-pip \
  liburdfdom-dev \
  liburdfdom-headers-dev \
  python3-dev \
  cmake \
  vim \
  git

# Build and install gsl from source
tar -xzf gsl.tar.gz
cd gsl
./configure CFLAGS="-g -O0" --prefix="/usr"
make -j4
sudo make install
```

```
# Build and install eigen3 from source
tar -xzf eigen-3.3.7.tar.gz
cd eigen-3.3.7
mkdir build
cd build
cmake ..
sudo make install
```

- Install DexHandSDK

```
# Linux C++ version installation
sudo dpkg -i DexHandSDK-3.0.0-Linux.deb
```

## 2.3 Troubleshooting

### Problem 1: Insufficient serial port permissions

Symptom: When connecting, it prompts **Permission denied: '/dev/ttyUSB0'**.

Solution:

```
# Confirm whether the device exists by re-plugging it
ls /dev/ttyUSB*

# Temporary authorization for the queried device (invalid after reboot); take
/dev/ttyUSB0 as an example
sudo chmod 666 /dev/ttyUSB0

# Permanent authorization (recommended)
sudo usermod -aG dialout $USER
# Log out and log back in for the change to take effect
```

### Problem 2: USB device not found

Symptom: **ls /dev/ttyUSB\*** has no output.

Solution:

```
# Check whether the USB device is recognized by the system
lsusb

# View the kernel log to confirm the connection status
dmesg | tail -20

# Load the USB serial driver (if it is not loaded automatically)
```

```
sudo modprobe cp210x      # Silicon Labs chip
sudo modprobe ch341       # CH340 / CH341 chip
```

### Problem 3: `scanModbusDevices()` scan times out or returns an empty list

Reasons and Handling: *(no additional content provided in the source document)*

---

## 3. Quick Start

### 3.1 Device Connection

Currently, DexHandSDK supports the Modbus communication protocol and the USB 2.0 communication protocol.

For the Modbus communication protocol, DexHandSDK provides two connection methods, as shown below.

| Modbus Auto Connection (yellow badge)                                                                        | Modbus Manual Connection (green badge)                                                            |
|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| Scenario: Connect via a USB-to-485 converter, no need to manually configure the baud rate and slave address. | Scenario: Given the baud rate and slave ID, precise control of connection parameters is required. |

#### Auto-connection workflow

The steps for using the automatic connection method of DexHandSDK Modbus are as follows.

**Flow:** Auto Connect → Other Operations → Close Modbus Serial Port

Sample code is as follows.

```
#include <dexh15/dexh15_control.h>
#include <iostream>

/**
 * @brief Modbus auto connection
 * @details Steps:
 * (1) Auto-connect to the first slave on the Modbus serial port
 * (2) Query the dexterous-hand online status
 * (3) Close the Modbus serial port
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
```

```

// Modbus serial port name
std::string hand_port_num = "/dev/ttyUSB0";
// Modbus slave address
int16_t device_address;
// Auto-connect to the first slave on the Modbus serial port
if (!dexh15->connectModbusAuto(hand_port_num, device_address))
{
    std::cerr << "Please check whether the dexterous hand is connected to the
Modbus serial device" << std::endl;
    return 1;
}
std::cout << "Auto-connection to the dexterous hand succeeded" << std::endl;
// Query the dexterous-hand online status
bool is_connected = dexh15->isModbusDeviceConnected(device_address);
std::cout << "The dexterous hand is " << (is_connected ? "online" : "offline")
<< std::endl;
// Close the Modbus serial port
bool result = dexh15->disconnectModbus();
std::cout << "Disconnect Modbus serial device " << hand_port_num
<< (result ? " succeeded" : " failed") << std::endl;
return 0;
}

```

## Manual-connection workflow

The steps for using the manual connection method of DexHandSDK Modbus are as follows.

**Flow:** Open Modbus Serial Port → Initialize Modbus Slave → Initialize Motor Position → Other Operations → Close Modbus Serial Port

Sample code is as follows.

```

#include <dexh15/dexh15_control.h>
#include <iostream>

/**
 * @brief Modbus manual connection
 * @details Steps:
 * (1) Open the Modbus serial port
 * (2) Initialize the Modbus slave
 * (3) Initialize the motor position (can be skipped if already called at power-
up)
 * (4) Query the dexterous-hand online status
 * (5) Close the Modbus serial port
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    // Modbus serial port name
    std::string hand_port_num = "/dev/ttyUSB0";

```

```

// Baud rate
int32_t baud_rate = 4000000;
// Modbus slave address
int16_t device_address = 0x79;
// Open the Modbus serial port
if (!dexh15->openModbusDevice(hand_port_num, baud_rate))
{
    std::cerr << "Please check whether a Modbus serial device is plugged in" <<
std::endl;
    return 1;
}
std::cout << "Open Modbus serial port succeeded" << std::endl;
// Initialize the Modbus slave
if (!dexh15->initModbusDevice(device_address))
{
    std::cerr << "Please check whether the dexterous hand is connected to the
Modbus serial device" << std::endl;
    dexh15->disconnectModbus();
    return 1;
}
std::cout << "Initialize Modbus slave succeeded" << std::endl;
// Initialize the motor position (can be skipped if already called at power-up)
if (!dexh15->initMotorPosition(device_address))
{
    std::cerr << "Initialize motor position failed" << std::endl;
    dexh15->disconnectModbus();
    return 1;
}
std::cout << "Initialize motor position succeeded" << std::endl;
// Query the dexterous-hand online status
bool is_connected = dexh15->isModbusDeviceConnected(device_address);
std::cout << "The dexterous hand is " << (is_connected ? "online" : "offline")
<< std::endl;
// Close the Modbus serial port
bool result = dexh15->disconnectModbus();
std::cout << "Disconnect Modbus serial device " << hand_port_num
    << (is_connected ? " succeeded" : " failed") << std::endl;
return 0;
}

```

## USB direct connection

For the USB 2.0 communication protocol, DexHandSDK provides a connection method, as shown below.

**USB Direct Connection** (blue badge) — Scenario: Single-handed debugging, academic research, and other scenarios.

The steps for using the DexHandSDK USB direct connection method are as follows.

**Flow:** Open USB Serial Port → Initialize Motor Position → Other Operations → Close USB Serial Port

Sample code is as follows.

```
#include <dexh15/dexh15_control.h>
#include <iostream>

/**
 * @brief USB direct connection
 * @details Steps:
 * (1) Open the USB serial port
 * (2) Initialize the motor position (can be skipped if already called at power-
up)
 * (3) Query the dexterous-hand online status
 * (4) Close the USB serial port
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    // USB serial port name
    std::string hand_port_num = "/dev/ttyACM0";
    // Slave address
    int16_t device_address = 0x79;
    // Open the USB serial port
    if (!dexh15->openUsbDevice(hand_port_num, device_address))
    {
        std::cerr << "Please check whether the USB serial device is plugged in" <<
std::endl;
        return 1;
    }
    std::cout << "Open USB serial port succeeded" << std::endl;
    // Initialize the motor position (can be skipped if already called at power-up)
    if (!dexh15->initMotorPosition(device_address))
    {
        std::cerr << "Initialize motor position failed" << std::endl;
        dexh15->disconnectUsb();
        return 1;
    }
    std::cout << "Initialize motor position succeeded" << std::endl;
    // Query the dexterous-hand online status
    bool is_connected = dexh15->isModbusDeviceConnected(device_address);
    std::cout << "The dexterous hand is " << (is_connected ? "online" : "offline")
<< std::endl;
    // Close the USB serial port
    bool result = dexh15->disconnectUsb();
    std::cout << "Disconnect USB serial device " << hand_port_num
        << (is_connected ? " succeeded" : " failed") << std::endl;
    return 0;
}
```

## 3.2 Typical Usage Process

This section provides 6 examples, which are:

| Position Mode (blue badge)       | Fault Diagnosis (green badge)                     | Sensor Data (gray badge)       |
|----------------------------------|---------------------------------------------------|--------------------------------|
| Predefined Actions (black badge) | Force-Position Hybrid Control Mode (purple badge) | Camera Function (yellow badge) |

### 3.2.1 Position Mode

The steps for using the position mode of DexHandSDK are as follows.

**Flow:** Open Modbus Serial Port → Initialize Modbus Slave → Initialize Motor Position → Set Position Mode → Enable Motor → Set Motor Position → Other Operations → Motor Return to Zero → Disable Motor → Close Modbus Serial Port

Sample code is as follows.

```
#include <dexh15/dexh15_control.h>
#include <dexh15/dexh15_define.h>

#include <iostream>
#include <cmath>
#include <sstream>
#include <thread>

/**
 * @brief Position mode
 * @details Steps:
 * (1) Open the Modbus serial port
 * (2) Initialize the Modbus slave
 * (3) Initialize the motor position (skip if already called at power-up)
 * (4) Query the dexterous-hand online status
 * (5) Set position mode
 * (6) Enable the motors
 * (7) Query the enabled status
 * (8) Set motor positions
 * (9) Return motors to zero
 * (10) Disable the motors
 * (11) Close the Modbus serial port
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    std::string hand_port_num = "/dev/ttyUSB0";
    int32_t baud_rate = 4000000;
    int16_t device_address = 0x79;
```



```

    if (!dexh15->openModbusDevice(hand_port_num, baud_rate))
    {
        std::cerr << "Please check whether a Modbus serial device is plugged in" <<
std::endl;
        return 1;
    }
    std::cout << "Open Modbus serial port succeeded" << std::endl;

    if (!dexh15->initModbusDevice(device_address))
    {
        std::cerr << "Please check whether the dexterous hand is connected to the
Modbus serial device" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    std::cout << "Initialize Modbus slave succeeded" << std::endl;

    if (!dexh15->initMotorPosition(device_address))
    {
        std::cerr << "Initialize motor position failed" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    std::cout << "Initialize motor position succeeded" << std::endl;

    bool is_connected = dexh15->isModbusDeviceConnected(device_address);
    std::cout << "Dexterous-hand connection status: " << (is_connected ? "online" :
"offline") << std::endl;

    // Set position mode
    if (!dexh15->setMotorControlMode(device_address,
                                     paxini::bot::dexh15::POSITION_CONTROL_MODE))
    {
        std::cerr << "Set position mode failed; cannot perform position-mode
control" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    std::cout << "Set position mode succeeded" << std::endl;

    // Enable motors
    if (!dexh15->enableMotor(device_address))
    {
        std::cerr << "Enable motor failed; cannot perform position-mode control" <<
std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    std::cout << "Enable motor succeeded" << std::endl;

    // Query enabled status
    if (!dexh15->isMotorEnabled(device_address))
    {

```

```

        std::cerr << "Motor is not enabled; cannot perform position-mode control"
<< std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    std::cout << "Motor is enabled" << std::endl;

    // Set motor positions
    paxini::bot::dexh15::Dex15MotorPosition positions{8000, 8000, 8000, 8000, 3000,
5000, 5000};
    if (!dexh15->setMotorTargetPosition(device_address, positions))
    {
        std::cerr << "Set motor position failed" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    std::cout << "Set motor position succeeded" << std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));

    // Return motors to zero
    paxini::bot::dexh15::Dex15MotorPosition zero_positions{0, 0, 0, 0, 0, 0, 0};
    if (!dexh15->setMotorTargetPosition(device_address, zero_positions))
    {
        std::cerr << "Return motors to zero failed" << std::endl;
    }
    bool is_success = dexh15->disableMotor(device_address);
    std::cout << "Disable motor " << (is_success ? "succeeded" : "failed") <<
std::endl;

    is_success = dexh15->disconnectModbus();
    std::cout << "Disconnect Modbus serial device " << hand_port_num
        << (is_success ? " succeeded" : " failed") << std::endl;
    return 0;
}

```

### 3.2.2 Fault Diagnosis

The steps for using DexHandSDK fault diagnosis are as follows.

**Flow:** Open Modbus Serial Port → Initialize Modbus Slave → Initialize Motor Position → Check Hand Status → Query Fault Code → Clear Fault Code → Close Modbus Serial Port

Sample code is as follows.

```

#include <dexh15/dexh15_control.h>
#include <iostream>

/**
 * @brief Fault diagnosis
 * @details Steps:
 *      (1) Open the Modbus serial port

```

```

* (2) Initialize the Modbus slave
* (3) Initialize the motor position (skip if already called at power-up)
* (4) Query the dexterous-hand online status
* (5) Check the dexterous-hand status
* (6) Query motor fault codes
* (7) Clear motor fault codes
* (8) Close the Modbus serial port
*/
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    std::string hand_port_num = "/dev/ttyUSB0";
    int32_t baud_rate = 4000000;
    int16_t device_address = 0x79;

    if (!dexh15->openModbusDevice(hand_port_num, baud_rate))
    {
        std::cerr << "Please check whether a Modbus serial device is plugged in" <<
std::endl;
        return 1;
    }
    std::cout << "Open Modbus serial port succeeded" << std::endl;

    if (!dexh15->initModbusDevice(device_address))
    {
        std::cerr << "Please check whether the dexterous hand is connected to the
Modbus serial device" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    std::cout << "Initialize Modbus slave succeeded" << std::endl;

    if (!dexh15->initMotorPosition(device_address))
    {
        std::cerr << "Initialize motor position failed" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    std::cout << "Initialize motor position succeeded" << std::endl;

    bool is_connected = dexh15->isModbusDeviceConnected(device_address);
    std::cout << "The dexterous hand is " << (is_connected ? "online" : "offline")
<< std::endl;

    int hand_status = dexh15->checkHandStatus(device_address);
    std::cout << "Current dexterous-hand status: "
        << (hand_status == 1 ? "Normal operating state"
            : hand_status == 2 ? "Fault-tolerant operating
state"
            : "Inoperable state")
        << std::endl;

    std::vector<paxini::bot::dexh15::FingerMotorsError> fault_codes =

```

```

        dexh15->getFaultCode(device_address);
    for (const auto &error : fault_codes)
    {
        std::cout << "Finger: " << error.finger_name
                    << ", Motor ID: " << error.motor_ID
                    << ", Error code: " << static_cast<int>(error.error_code)
                    << std::endl;
    }

    bool result = dexh15->clearFaultCode(device_address);
    std::cout << "Clear motor fault codes " << (result ? "succeeded" : "failed") <<
std::endl;

    result = dexh15->disconnectModbus();
    std::cout << "Disconnect Modbus serial device " << hand_port_num
                << (result ? " succeeded" : " failed") << std::endl;
    return 0;
}

```

### 3.2.3 Sensor Data

The steps for using DexHandSDK sensor data are as follows.

**Flow:** Open Modbus Serial Port → Initialize Modbus Slave → Calibrate Sensor → Read Sensor Data → Close Modbus Serial Port

Sample code is as follows.

```

#include <dexh15/dexh15_control.h>

#include <sstream>
#include <thread>

/**
 * @brief Read sensor data
 * @details Steps:
 *   (1) Open the Modbus serial port
 *   (2) Initialize the Modbus slave
 *   (3) Calibrate the sensors
 *   (4) Read the resultant sensor force
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    std::string hand_port_num = "/dev/ttyUSB0";
    int32_t baud_rate = 4000000;
    int16_t device_address = 0x79;

    if (!dexh15->openModbusDevice(hand_port_num, baud_rate))
    {

```

```

        std::cerr << "Please check whether a Modbus serial device is plugged in" <<
std::endl;
        return 1;
    }
    std::cout << "Open Modbus serial port succeeded" << std::endl;

    if (!dexh15->initModbusDevice(device_address))
    {
        std::cerr << "Please check whether the dexterous hand is connected to the
Modbus serial device" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    std::cout << "Initialize Modbus slave succeeded" << std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));

    if (!dexh15->calibrateSensor(device_address))
    {
        std::cerr << "Calibrate sensor failed" << std::endl;
    }
    std::cout << "Calibrate sensor succeeded" << std::endl;

    // Acquire sensor data (values change only when the fingertip sensor is
pressed)
    int hz = 10;
    while (true)
    {
        std::vector<paxini::bot::dexh15::ForcePoint> force_points =
            dexh15->getFingerTactile(device_address);
        for (auto force_point : force_points)
        {
            std::cout << "x:" << force_point.x
                << ", y:" << force_point.y
                << ", z:" << force_point.z << std::endl;
        }
        std::this_thread::sleep_for(
            std::chrono::milliseconds(static_cast<int>(1000.0 / hz)));
    }
    return 0;
}

```

### 3.2.4 Predefined Actions

The steps for using the predefined actions of DexHandSDK are as follows.

**Flow:** Open Modbus Serial Port → Initialize Modbus Slave → Initialize Motor Position → Set Position Mode → Enable Motor → Run Predefined Action → Disable Motor → Close Modbus Serial Port

Sample code is as follows.

```

#include <dexh15/dexh15_control.h>
#include <dexh15/dexh15_define.h>

#include <iostream>
#include <cmath>
#include <sstream>
#include <thread>

/**
 * @brief Run predefined actions
 * @details Steps:
 * (1) Open the Modbus serial port
 * (2) Initialize the Modbus slave
 * (3) Initialize the motor position (skip if already called at power-up)
 * (4) Query the dexterous-hand online status
 * (5) Set position mode
 * (6) Enable the motors
 * (7) Query the enabled status
 * (8) Run predefined actions 1 to 10
 * (9) Disable the motors
 * (10) Close the Modbus serial port
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    std::string hand_port_num = "/dev/ttyUSB0";
    int32_t baud_rate = 4000000;
    int16_t device_address = 0x79;

    if (!dexh15->openModbusDevice(hand_port_num, baud_rate))
    {
        std::cerr << "Please check whether a Modbus serial device is plugged in" <<
std::endl;
        return 1;
    }
    std::cout << "Open Modbus serial port succeeded" << std::endl;

    if (!dexh15->initModbusDevice(device_address))
    {
        std::cerr << "Please check whether the dexterous hand is connected to the
Modbus serial device" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
    std::cout << "Initialize Modbus slave succeeded" << std::endl;

    if (!dexh15->initMotorPosition(device_address))
    {
        std::cerr << "Initialize motor position failed" << std::endl;
        dexh15->disconnectModbus();
        return 1;
    }
}

```

```

}
std::cout << "Initialize motor position succeeded" << std::endl;

bool is_connected = dexh15->isModbusDeviceConnected(device_address);
std::cout << "Dexterous-hand connection status: " << (is_connected ? "online" :
"offline") << std::endl;

if (!dexh15->setMotorControlMode(device_address,
                                paxini::bot::dexh15::POSITION_CONTROL_MODE))
{
    std::cerr << "Set position mode failed; cannot perform position-mode
control" << std::endl;
    dexh15->disconnectModbus();
    return 1;
}
std::cout << "Set position mode succeeded" << std::endl;

if (!dexh15->enableMotor(device_address))
{
    std::cerr << "Enable motor failed; cannot perform position-mode control" <<
std::endl;
    dexh15->disconnectModbus();
    return 1;
}
std::cout << "Enable motor succeeded" << std::endl;

if (!dexh15->isMotorEnabled(device_address))
{
    std::cerr << "Motor is not enabled; cannot perform position-mode control"
<< std::endl;
    dexh15->disconnectModbus();
    return 1;
}
std::cout << "Motor is enabled" << std::endl;

paxini::bot::dexh15::Dex15MotorPosition zero_positions{0, 0, 0, 0, 0, 0, 0};
// Run built-in actions; there are 10 in total
for (int i = 1; i <= 10; i++)
{
    if (!dexh15->runAction(device_address, i, false))
    {
        std::cerr << "Run built-in action failed, id = " << std::to_string(i)
<< std::endl;
    }
    std::cout << "Run built-in action succeeded, id = " << std::to_string(i) <<
std::endl;
    std::this_thread::sleep_for(std::chrono::seconds(1));
    dexh15->setMotorTargetPosition(device_address, zero_positions);
    std::this_thread::sleep_for(std::chrono::seconds(1));
}

bool is_success = dexh15->disableMotor(device_address);
std::cout << "Disable motor " << (is_success ? "succeeded" : "failed") <<
std::endl;

```

```

    is_success = dexh15->disconnectModbus();
    std::cout << "Disconnect Modbus serial device " << hand_port_num
               << (is_success ? " succeeded" : " failed") << std::endl;
    return 0;
}

```

### 3.2.5 Force-Position Hybrid Control Mode

The steps for using the force-position hybrid control mode of DexHandSDK are as follows.

**Flow:** Open Modbus Serial Port → Initialize Modbus Slave → Initialize Motor Position → Set Position Mode → Enable Motor → Set Normal-Force Force-Control Parameters → Compute Next Target Angle ⇌ Set Next Target Angle (loop) → Disable Motor → Close Modbus Serial Port

Sample code is as follows.

```

#include <dexh15/dexh15_control.h>

#include <iostream>
#include <chrono>
#include <thread>

/**
 * @brief Normal-force force-position hybrid control mode
 * @details Steps:
 *   (1) Open the Modbus serial port
 *   (2) Initialize the Modbus slave
 *   (3) Initialize the motor position (skip if already called at power-up)
 *   (4) Query the dexterous-hand online status
 *   (5) Set position mode
 *   (6) Enable the motors
 *   (7) Query the enabled status
 *   (8) Set normal-force force-control parameters
 *   (9) Compute the target angle for the next motion
 *   (10) Set the target angle for the next motion
 *   (11) Repeat (9) and (10) until the object is gripped and the normal force is
constant
 *   (12) Disable the motors
 *   (13) Close the Modbus serial port
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Control> dexh15 =
        std::make_shared<paxini::bot::dexh15::DexH15Control>();
    std::string hand_port_num = "/dev/ttyUSB0";
    int32_t baud_rate = 4000000;
    int16_t device_address = 0x79;

    if (!dexh15->openModbusDevice(hand_port_num, baud_rate))

```



```

{
    std::cerr << "Please check whether a Modbus serial device is plugged in" <<
std::endl;
    return 1;
}
std::cout << "Open Modbus serial port succeeded" << std::endl;

if (!dexh15->initModbusDevice(device_address))
{
    std::cerr << "Please check whether the dexterous hand is connected to the
Modbus serial device" << std::endl;
    dexh15->disconnectModbus();
    return 1;
}
std::cout << "Initialize Modbus slave succeeded" << std::endl;

if (!dexh15->initMotorPosition(device_address))
{
    std::cerr << "Initialize motor position failed" << std::endl;
    dexh15->disconnectModbus();
    return 1;
}
std::cout << "Initialize motor position succeeded" << std::endl;

bool is_connected = dexh15->isModbusDeviceConnected(device_address);
std::cout << "Dexterous-hand connection status: " << (is_connected ? "online" :
"offline") << std::endl;

if (!dexh15->setMotorControlMode(device_address,
                                paxini::bot::dexh15::POSITION_CONTROL_MODE))
{
    std::cerr << "Set position mode failed; cannot perform position-mode
control" << std::endl;
    dexh15->disconnectModbus();
    return 1;
}
std::cout << "Set position mode succeeded" << std::endl;

if (!dexh15->enableMotor(device_address))
{
    std::cerr << "Enable motor failed; cannot perform position-mode control" <<
std::endl;
    dexh15->disconnectModbus();
    return 1;
}
std::cout << "Enable motor succeeded" << std::endl;

if (!dexh15->isMotorEnabled(device_address))
{
    std::cerr << "Motor is not enabled; cannot perform position-mode control"
<< std::endl;
    dexh15->disconnectModbus();
    return 1;
}
}

```

```

std::cout << "Motor is enabled" << std::endl;

// Normal-force control parameters
paxini::bot::dexh15::ForceCtlParam force_ctl_param{
    {50, 50, 50, 50, 50, 50, 50, 50, 50, 50},
    {5, 5, 5, 5, 5, 5, 5, 5, 5, 5},
    {true, true, true, true, true, true, true, true, true, true}};
// Initial pose
std::vector<double> original_angle = {0, 0, 0, 0, 0, 0, 0, 0, 500, 0, 0};

if (!dexh15->setCommonForceCtlParam(device_address, original_angle,
force_ctl_param))
{
    std::cerr << "Set normal-force force-control parameters failed" <<
std::endl;
    dexh15->disconnectModbus();
    return 1;
}
std::cout << "Set normal-force force-control parameters succeeded" <<
std::endl;

// Run normal-force force-position hybrid control
auto start_time = std::chrono::steady_clock::now();
auto duration = std::chrono::seconds(30);
while (std::chrono::steady_clock::now() - start_time < duration)
{
    std::vector<double> command;
    if (!dexh15->computeCommonForceTargetCommand(device_address, command))
    {
        std::cerr << "Compute next target angle failed" << std::endl;
        break;
    }
    std::cout << "Compute next target angle succeeded" << std::endl;
    if (!dexh15->setJointPositionsAngle(device_address, command))
    {
        std::cerr << "Set next target angle failed" << std::endl;
        break;
    }
    std::cout << "Set next target angle succeeded" << std::endl;
    std::this_thread::sleep_for(std::chrono::milliseconds(5));
}

// Return motors to zero
paxini::bot::dexh15::Dex15MotorPosition zero_positions{0, 0, 0, 0, 0, 0, 0};
if (!dexh15->setMotorTargetPosition(device_address, zero_positions))
{
    std::cerr << "Return motors to zero failed" << std::endl;
}
std::cout << "Return motors to zero succeeded" << std::endl;

bool is_success = dexh15->disableMotor(device_address);
std::cout << "Disable motor " << (is_success ? "succeeded" : "failed") <<
std::endl;

```

```

    bool result = dexh15->disconnectModbus();
    std::cout << "Disconnect Modbus serial device " << hand_port_num
               << (is_connected ? " succeeded" : " failed") << std::endl;
    return 0;
}

```

### 3.2.6 Camera Function

The steps for using the camera function of DexHandSDK are as follows.

**Flow:** Open Camera Serial Port → Set Resolution and Frame Rate → Get a Frame → Display Image → Release Camera Serial Port

Sample code is as follows.

```

#include <dexh15/dexh15_camera.h>

#include <iostream>
#include <opencv2/opencv.hpp>

/**
 * @brief Camera function
 * @details Steps:
 * (1) Open the camera serial port
 * (2) Set resolution and frame rate
 * (3) Get one frame of image
 * (4) Display the image
 * (5) Release the camera serial port
 */
int main(int argc, char **argv)
{
    std::shared_ptr<paxini::bot::dexh15::DexH15Camera> dexh15_camera =
        std::make_shared<paxini::bot::dexh15::DexH15Camera>();
    std::string camera_port_num = "/dev/video4";
    try
    {
        dexh15_camera->connectCameraDevice(camera_port_num);

        if (!dexh15_camera->setCameraConfig(640, 480, 30))
        {
            std::cout << "Set camera parameters failed" << std::endl;
        }
        std::cout << "Set camera parameters succeeded" << std::endl;

        cv::Mat frame = dexh15_camera->getFrame();
        dexh15_camera->showImage("img", frame);

        dexh15_camera->releaseCameraDevice();
        std::cout << "Release camera serial port" << std::endl;
    }
    catch (...)

```

```
{  
}  
return 0;  
}
```

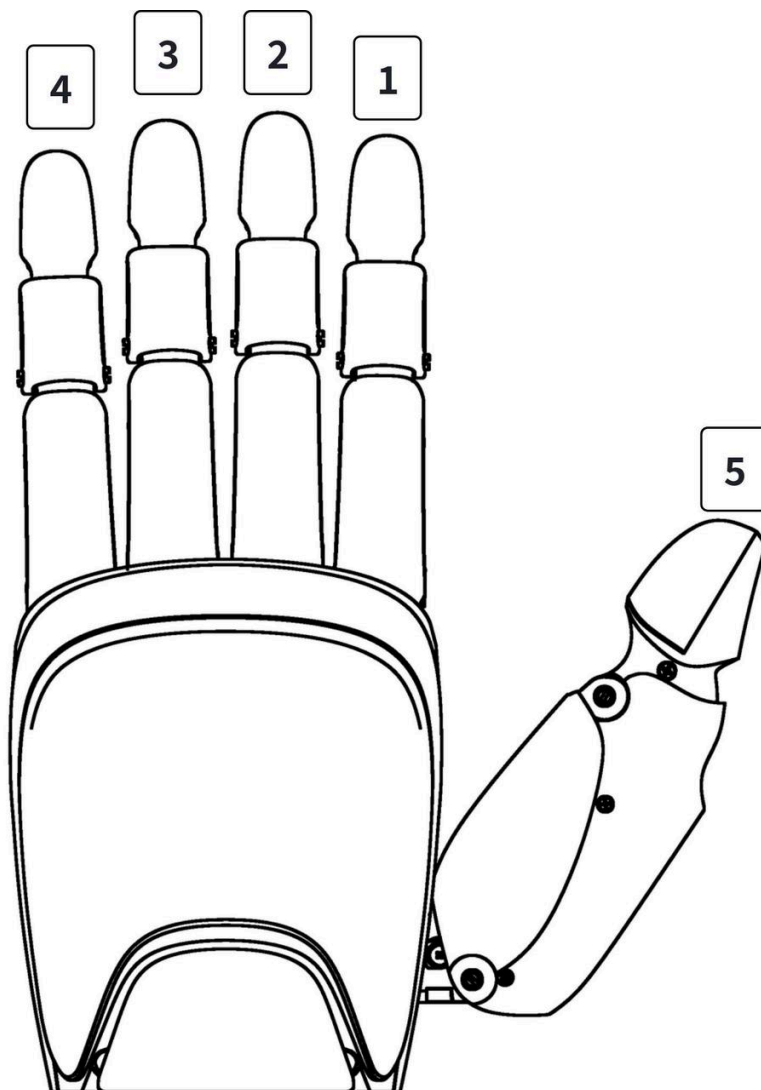
## 4. Core Concepts

This section introduces the key concepts you need to understand before using the SDK. Understanding these concepts will help you use each API more efficiently.

### 4.1 Hierarchical Addressing and Resource Mapping

#### 4.1.1 Finger Numbering

The GMH15 dexterous hand has a total of 5 fingers, and the finger numbering method is as follows.

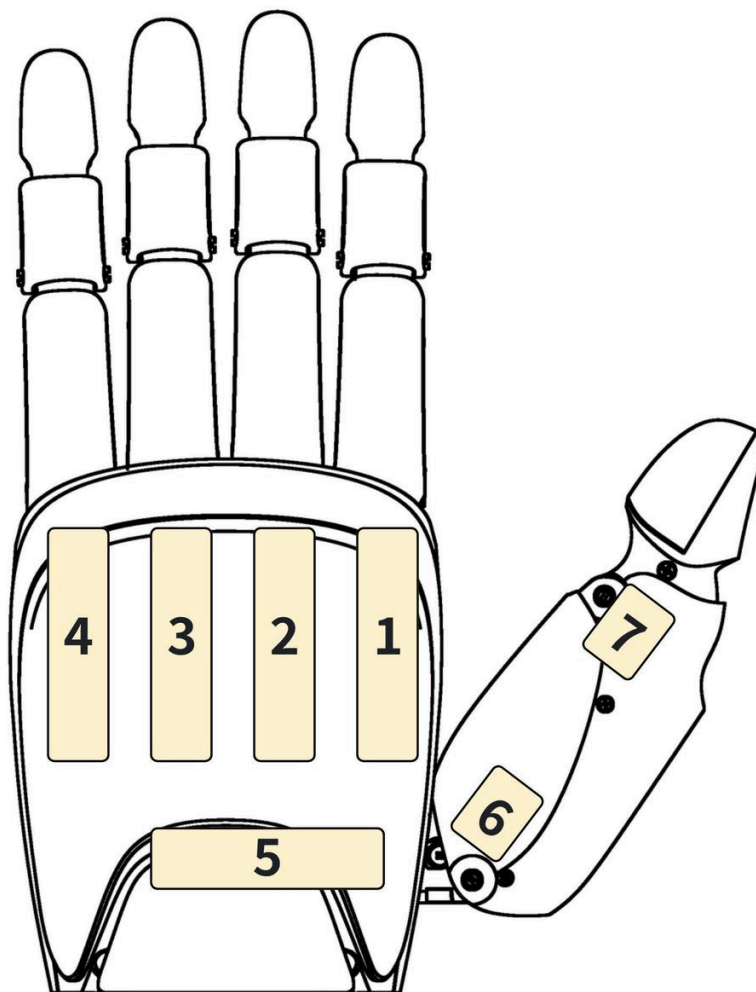


Legend:

- **Finger 1** — Index finger (closest to the thumb among the four long fingers)
- **Finger 2** — Middle finger
- **Finger 3** — Ring finger
- **Finger 4** — Little finger (outer-most long finger)
- **Finger 5** — Thumb (shown to the side, opposable)

#### 4.1.2 Motor Number

The GMH15 dexterous hand has a total of 7 motors, and the motor numbering method is as follows.

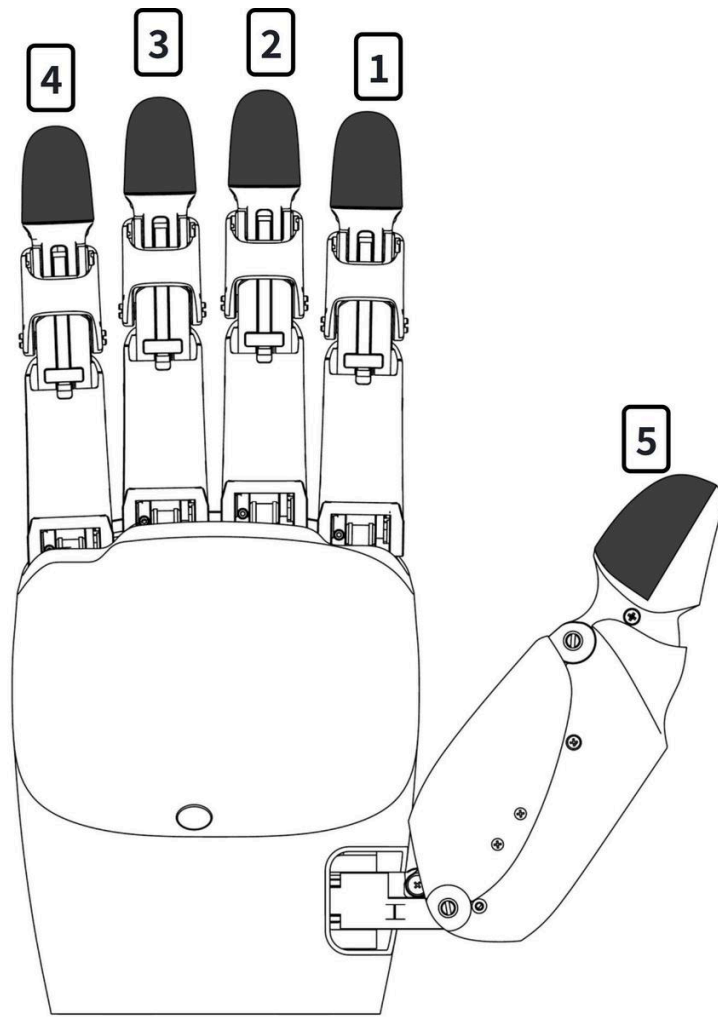


#### Legend:

- **Motor 1** — Index-finger flexion motor (palm)
- **Motor 2** — Middle-finger flexion motor (palm)
- **Motor 3** — Ring-finger flexion motor (palm)
- **Motor 4** — Little-finger flexion motor (palm)
- **Motor 5** — Common motor at the base of the four fingers (palm, horizontal bar)
- **Motor 6** — Thumb motor (lower joint of the thumb)
- **Motor 7** — Thumb motor (upper joint of the thumb)

#### 4.1.3 Sensor Number

The GMH15 dexterous hand has a total of 5 tactile sensor modules, and its finger numbering method is as follows.



#### Legend:

- **Sensor 1** — Index-finger fingertip
- **Sensor 2** — Middle-finger fingertip
- **Sensor 3** — Ring-finger fingertip
- **Sensor 4** — Little-finger fingertip
- **Sensor 5** — Thumb fingertip

## 4.2 **slave\_id** — Device Addressing

**slave\_id** is the first parameter of all direct control APIs in the SDK, used to address a specific device on the shared bus.

### 4.2.1 Value Normalization

The SDK uses **normalized relative values** instead of hardware absolute values to shield the parameter differences of different batches of hardware.

### 4.2.2 Control Mode

The dexterous hand supports three underlying motor control modes, suitable for different scenarios:

**⚠ Caution:** Before switching the control mode, it is recommended to first return the hand to the zero position (`resetToZeroPosition`) to avoid unexpected movement at the moment of mode switching.

---

## 5. Frequently Asked Questions

### 5.1 Connection Issue

#### 5.1.1 Modbus manual connection failed

**Reason:** Baud rate error, or slave address error.

**Solution:** Connect to GMH15 using the Modbus automatic connection method.

```
/**
 * @brief Auto-connect to the first dexterous-hand device on the current serial
 * port; suitable for single-device scenarios
 * @param hand_port_num: dexterous-hand serial-port name
 * @param device_address: returned address of the connected device
 * @return true on success, false on failure; throws on program error
 */
bool connectModbusAuto(const std::string& hand_port_num, int16_t& device_address);
```

### 5.2 Control Mode Setting Issue

#### 5.2.1 Failed to set control mode

**Reason:** Set the control mode while the motor is in the enabled state.

**Solution:** Disable the motor using the following method, then set the control mode.

```
/**
 * @brief Disable the motor
 * @param slave_address: slave address
 * @return true if the motor is disabled successfully, false otherwise
 */
bool disableMotor(int16_t slave_address);
```

### 5.3 Motor Control Issues

### 5.3.1 Failed to set motor position

**Reason:** The current state of the dexterous hand is non-operational, the motor is not enabled, or the control mode is not position mode.

#### Solution:

- **The current state of the dexterous hand is non-operational.** Use Section 3.2.2 fault diagnosis for preliminary investigation. If there is a motor disconnection error code, please contact our company (<https://www.paxini.com/>) for maintenance.
- **Motor is not enabled.** Enable the motor using the following method, then set the motor position.

```
/**
 * @brief Enable the motor
 * @param slave_address: slave address
 * @return true on success, false on failure
 */
bool enableMotor(int16_t slave_address);
```

- **Control mode is not position mode.** Set the control mode to position mode using the following method, then set the motor position.

```
/**
 * @brief Define the motor control mode; the default is position mode.
 * @param mode: motor control mode – current mode, velocity mode, or position
mode
 * @return true on success, false on failure
 */
bool setMotorControlMode(int16_t slave_address, const ControlMode mode);
```

### 5.3.2 Failed to set the motor target current

**Reason:** The motor target current is greater than the maximum allowable current of the motor, the motor is not enabled, or the control mode is not torque mode.

#### Solution:

- **Motor is not enabled.** Enable the motor using the following method, then set the motor target current.

```
/**
 * @brief Enable the motor
 * @param slave_address: slave address
```



```

    * @return true on success, false on failure
    */
    bool enableMotor(int16_t slave_address);

```

- **Control mode is not torque mode.** Set the control mode to torque mode using the following method, then set the motor target current.

```

/**
 * @brief Define the motor control mode; the default is position mode.
 * @param mode: motor control mode – current mode, velocity mode, or position
 mode
 * @return true on success, false on failure
 */
    bool setMotorControlMode(int16_t slave_address, const ControlMode mode);

```

- **The motor target current is greater than the maximum current allowed by the motor.** Query the maximum allowable current of the motor using the following method:

```

/**
 * @brief Get the maximum-current configuration of all motors of the slave
 device
 * @param slave_address: slave address
 * @return maximum-current configuration of all motors
 */
    Dex15MotorCurrent getMotorMaxCurrentConfig(int16_t slave_address);

```

If the maximum current allowed by the motor is less than the target current of the motor, modify the maximum current of the motor using the following method:

```

/**
 * @brief Define the maximum motor current
 * @param max_current: range [0, 1000]
 * @return true on success, false on failure
 */
    bool setMotorMaxCurrent(int16_t slave_address, const Dex15MotorCurrent&
 max_current);

```

## 5.4 Sensor Data Issues

### 5.4.1 When there is no pressing, the resultant force of the sensor is not zero

**Reason:** Uncalibrated sensor.

**Solution:** Before reading sensor data, calibrate the sensor using the following method.

```
/**
 * @brief Calibrate the dexterous-hand tactile sensors
 * @param slave_address: slave address
 * @return true on success, false on failure
 */
bool calibrateSensor(int16_t slave_address);
```

## 5.5 Image Processing Issues

### 5.5.1 Unable to recognize the camera serial port number

**Reason:** Type-C cable is damaged.

**Solution:** Check whether the Type-C cable is in good condition. If the Type-C cable is damaged, please replace it.

### 5.5.2 The acquired image is a black screen

**Reason:** Camera wiring is loose or damaged.

**Solution:** Please contact our company (<https://www.paxini.com/>) for repair.

---

## 6. Precautions

1. Before using the SDK, please ensure that the SDK environment has been correctly installed and that you have understood the basic operations and usage methods of the dexterous-hand product.
  2. When using the SDK, please call functions according to the instructions of the function interfaces to avoid issues such as parameter errors or incorrect call order.
  3. When controlling the movement of the dexterous hand, please set reasonable motion parameters according to the actual situation to avoid problems such as motion out of control or collisions.
  4. When acquiring sensor data, please set a reasonable data-acquisition frequency according to the actual situation to avoid issues such as data loss or delay.
  5. When using the SDK, if you encounter any issues or errors, please refer to the error codes and error messages for troubleshooting and resolution.
- 

## 7. Copyright Notice

1. This document will be regularly checked and revised, and the updated content will appear in the new version. Any changes to the content or information in this document will not be notified separately.

2. Please read this document before using the product.
3. Please keep this document well for easy reading and reference at any time.
4. The content recorded in this document does not exclude the possibility of errors or omissions. If you have any questions about the content in this document, please contact our company (<https://www.paxini.com/>).